

What we built, in plain words

No jargon. Read it top to bottom and you will be able to explain the whole thing to anyone.

PART 1

What is this thing?

You get a lot of email. Most of it does not need you.

Custodian reads your mailbox and tells you which few emails actually need you. Then it writes the replies for you, in your own writing style — and it asks before sending anything.

That is the whole product. Everything else is detail.

PART 2

What happens when an email arrives

Think of a person sorting post at a desk. They pick up each envelope and ask a series of questions. **Cheap questions first, expensive ones last.**

1 Should I even open this?

It looks only at *who sent it* and *the subject line*. If it is your bank, or a security code, or a medical result — **it never opens the envelope at all.** It just tells you it arrived.

2 Is this even from a person?

Companies put a hidden mark in their emails that says *"a machine sent this"*. We read that mark. **We don't guess.**

It also checks: *have you ever opened mail from this sender?* Measured on your account — one sender had sent **357 emails and you never opened a single one.**

About 9 out of 10 emails stop here. Cost so far: nothing.

3 Now open it and clean it up

A LinkedIn email is around **200,000 characters** — nearly all of it invisible formatting code and old quoted conversation.

After cleaning: **about 1,000 characters** of real words. That is 200 times smaller.

4 Read it

Now a model reads the words and answers one question: **is this person asking me for something, or just telling me something?**

This model is **ours**. It runs on your laptop. It costs nothing and the email never leaves your machine.

5 Does it need YOU?

A different question. A newsletter can ask a question; your boss can send one line with no question mark.

So a second model looks at *your history with this sender* — do you reply to them, do you open them, are you the only recipient. It gives one number.

6 Write the reply

Only now does anything cost money. A rented model writes a draft in your writing style — and it **never sees real names**, only placeholders.

7 Stop and ask

It shows you the draft and **waits**. Type yes and it sends. Type anything else and nothing happens.

The one idea behind all of it

Cheap questions first. Expensive ones last. Nine emails in ten never reach a model, so they cost nothing.

PART 3

What we keep, and what we don't

This is the part that confuses everyone. Here it is as a picture.



We copied the writing **off the outside** of 7,393 envelopes. We only opened 11 of them.

Why only 11?

Those 11 are recent emails from real people. The rest are job alerts, LinkedIn notifications and bounce messages. **We know they exist. We did not bother reading them.**

What if you want to read an old one?

We ask Gmail for it, right then. It takes **0.7 seconds**. Then we forget it again.

Gmail is already storing your mail. Keeping a second copy would only save us that one second — and cost us storage for ten years of email you will never reopen.

Why this matters commercially

We can honestly tell a customer: **"we don't hold your email."** No competitor can say that. And when someone says "delete everything about me", it takes us seconds instead of a compliance project.

The screen — what the list actually shows

```
python mailbox.py
```

7,393 headers held · 11 bodies cached (0.01 MB)

1 ○	OET Flash Sale: 80% OFF	E2Language	2026-09-10
2 ○	AI Solutions Associate	LinkedIn	2026-09-10
4 ○	Security alert	Google	2026-09-09
8 ●	Re: test	Feroz	2026-09-07
9 ●	Re: test	Feroz	2026-09-07
21 ○	Delivery Status Notification	Mail Daemon	2026-09-05

● GREEN DOT

We have the message text. Opens instantly.

Real emails from real people — the 5 replies from Feroz.

○ EMPTY CIRCLE

We only have the subject and sender.

Job alerts, notifications, bounce messages. Click one and it arrives in 0.7 seconds.

To read email number 8:

```
python mailbox.py --open 8
```

Instant — we have it.

To read email number 21:

```
python mailbox.py --open 21
```

Takes 0.7 seconds — we fetch it from Gmail, show it, and forget it.

And number 4 will refuse to open

"Security alert" from Google is exactly the kind of mail we promised never to read. It contains device names and locations. **The system will tell you it exists and refuse to show it.** That is the rule working, not a bug.

What changed after sir's meeting

Three things. That's all.

Change 1 — a free way into Gmail

There are two doors into a mailbox.

THE API — OLD WAY

Google charges for a **security audit every year**. Plus months of waiting for approval. Plus you can only have 100 users until it clears.

IMAP — NEW WAY

A 16-letter password from Google.
Free. No approval. Works today.

Every command now takes `--imap` to use the free door. **Nothing else changed** — same models, same steps.

Change 2 — sir's question, answered

He asked: *Gmail has tabs — Primary, Social, Promotions. Can we get only Primary through IMAP?*

Everyone assumed no. IMAP is a 1990s protocol with no idea what a tab is.

But Gmail's IMAP has a hidden feature that lets us type Gmail search queries. So we ask for the Primary tab and get it.

<code>category:primary</code>	1,747
<code>category:social</code>	2,834
<code>category:promotions</code>	1,260
<code>category:updates</code>	3,554

Answer: yes. 1,747 emails. On your own mailbox, not in theory.

Change 3 — the mailbox screen

Umar described it: headers for everything, message text only for the last 30 days, and only the important ones. Click an old one and it loads in two seconds.

That is now `python mailbox.py`, and it loads in 0.7 seconds rather than two.

PART 6

Four bugs found by running it on real mail

None of these crashed. All four gave answers that *looked completely normal*.

1 • Nothing was in date order

Dates were stored as plain text like "Wed, 9 Jun 2021". Sorting that put **2021 next to 2025**, because both start with "Wed, 9".

So every "newest emails first" list in the whole system was actually alphabetical. **Found because a command that should have pulled 40 recent emails pulled zero** — it was looking at 2021.

2 • Security emails were being read

The rules blocked "*reset your password*" but not "*your password **has been changed***".

So the *request* was blocked and the *confirmation* was not — and the confirmation is the one that proves somebody took over your account.

3 • Bounce messages were being stored

Seven "Delivery Status Notification (Failure)" messages had their text saved, because of a small mistake: the code that decides what to keep was not being given the field that identifies machine mail. So every message looked human-sent.

4 • A sale advert created a fake deadline

A marketing email said *"80% off today"*. The system saw the word "today", decided it was a deadline, and set a reminder — **for five minutes after the deadline had already passed**.

Fixed: bulk marketing mail can no longer create reminders from its words. It still can from an attached calendar invite, because that is the sender declaring a real appointment.

The pattern worth telling sir

The dangerous bugs here don't throw errors. They give confident wrong answers. You only find them by looking at real output and asking "should that be there?"

PART 7

Every command, and what it does

TYPE THIS	AND YOU GET
<code>python mailbox.py</code>	Your mail, in a list. The main screen.
<code>python mailbox.py --open 8</code>	Read email number 8
<code>python agent.py 5</code>	Watch it think — every step, with reasons
<code>python agent.py 5 --imap</code>	The same, through the free door
<code>python test_imap.py</code>	Proof for sir — the Primary tab numbers
<code>python brief.py</code>	What needs you today
<code>python watch.py</code>	Leave it running — reacts to new mail
<code>python chat.py</code>	Talk to it in sentences
<code>python correct.py</code>	Tell it it was wrong
<code>python history.py</code>	Everything it did, and undo

All of them run in PowerShell, in the folder `c:\mob_ai\poc`

What to say to sir

"I built the IMAP version. Same code — one flag switches the door. No yearly audit, no approval, no fee."

"Your question is answered: **yes, IMAP can give us only the Primary tab. 1,747 emails** on my account."

"And I built the mailbox screen. We keep the subject and sender of all 7,393 emails, but the actual text of only 11. Old ones load in under a second when you click."

"I also found four bugs by running it on real mail, and fixed them."

If he asks: why IMAP?

No yearly security audit, no months of approval, no fee. And it is the **only** way into Yahoo, iCloud and company mailboxes — which no competitor serves.

If he asks: did the models change?

No. Same three models, same steps. **One new file** — the IMAP connector. Everything above it is untouched, and I can show the identical output through both doors.

If he asks: what is left?

Outlook and JMAP connectors — he said Google only for now. Each is one file. And a real screen instead of typed commands.